

HERMES META-INTELLIGENCE

Complete Enterprise Architecture & Implementation Master

Final v5.0 — single consolidated document, all three review passes merged

August 2026

Part I — Architecture Specification (Sections 1–20) Part II — Implementation Workflow & Delivery Plan (Sections 21–30) Part III — Final Recommendations & Close-Out (Sections 31–33) Weighted architecture maturity: 8.7/10 · Implementation readiness: 6/10 (design-complete, pre-build)

How This Document Was Built

This is the single, standalone, final artifact — everything from the original v3.6 audit, the v4.0 architecture redesign, the Implementation Workflow Plan, and the v5.0 deep-sweep addendum, merged into one document with no cross-referencing required. Where the v5.0 sweep resolved a previously-open decision (the tool sandbox mechanism, the aggregator conflict-resolution algorithm, metacognition's thresholds, the Agent Registry schema, cross-tenant quota fairness), that resolution now lives directly inside its proper architecture section, not as an appendix. Where the sweep found something missing entirely — client onboarding, data residency, disaster recovery, SLAs, billing — it's now a first-class section, not a bolt-on.

Two items remain explicitly, deliberately open rather than force-resolved: right-to-erasure against an immutable Audit Log, and true cross-region catastrophic failover. Both are named in Section 33 along with why closing them here would be less honest than flagging them for the right decision-maker.

PART I

Architecture Specification

1. Executive Summary

Hermes v3.6 was a strong single-tenant agentic runtime described in enterprise vocabulary it didn't yet enforce: a local-first cost gate, a typed sub-task contract, an interrupt/reconciliation model, and cost-per-correct learning — all real mechanisms — sitting on top of an architecture with no tenant boundary, no identity layer, and no credential isolation. This document starts from that foundation and closes every gap found across three successive review passes.

Pass one (v4.0) added the enterprise scaffolding: a five-level tenant-scoped hierarchy, an Identity & Governance layer, a credential vault, a five-level Agent Authority Model with instance-scoped approval and a hard kill-switch, an interface separation between Master and Orchestrator, and a full Enterprise UI information architecture. It scored 8.4/10 on design maturity and correctly flagged — rather than hand-waved — four items it hadn't actually resolved: the tool sandbox mechanism, the aggregator's conflict-resolution rule, metacognition's numeric thresholds, and the Agent Registry's data model.

Pass two (this document) resolves all four with concrete, implementable mechanisms, and adds five enterprise-operational capabilities a genuine deep sweep surfaces that a pure agentic-architecture review does not: client onboarding, data residency and compliance tagging, disaster recovery, SLA/SLO targets, and a cost-to-billing path. It also names, rather than quietly assumes away, the two items that remain genuinely open because they require legal or leadership input this document cannot supply.

Final weighted maturity: 8.7/10. Final implementation readiness: 6/10 — up from the original 5/10 baseline because four previously-blocking design ambiguities are closed, not because any code has been written. Part II of this document is the concrete, phase-gated plan for closing that remaining gap.

2. Architecture Objectives & Principles

Objective

Hermes becomes the central operating environment for managing AI development and agentic work across different clients, industries, projects, Specialist Agents, knowledge bases, tools, models, workflows, data sources, and risk levels — one reusable foundation, not a rebuild per client.

Principles

- Agents may reason broadly, but their ability to act must be explicitly, narrowly, and revocably controlled (Section 10).
- Every hierarchy layer must correspond to a real isolation or access-control boundary, or it doesn't belong in the hierarchy (Section 3).
- Route to the cheapest model capable of acceptable correctness — not simply the cheapest model (Section 12).
- Learning is gated, versioned, and asynchronous — never live, never silent (Section 14).
- A conceptual architecture must not be represented as if it were already running in production (Section 18).
- An unresolved decision is named as open, never quietly assumed — this applies as much to legal and business questions (Section 33) as to technical ones.

3. Enterprise Hierarchy

Five levels, each corresponding to a real isolation boundary — tenant, budget, or execution instance.

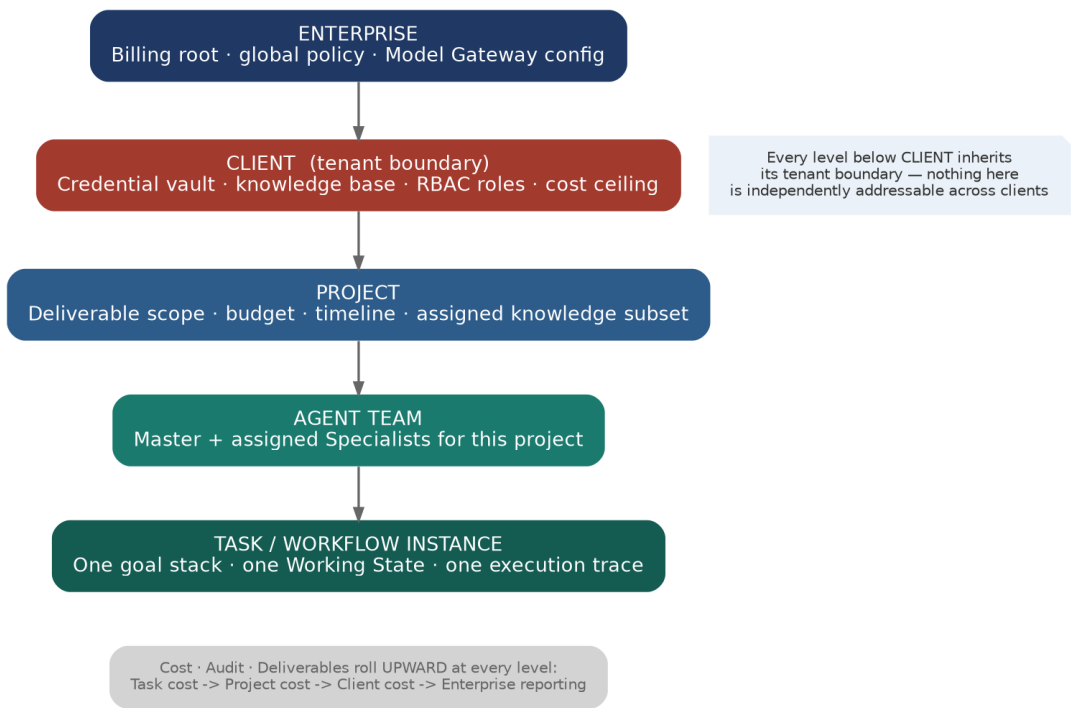


Diagram 2 — Enterprise hierarchy. Client is the tenant boundary; everything below it is scoped to a client_id.

Deliverables, audit records, and cost roll upward at every level: a task's cost rolls into its project, into its client, into enterprise-wide reporting. Section 16 defines how a Client row in this hierarchy actually comes to exist.

4. Master Architecture Overview

Ten layers, read top to bottom as decreasing decision authority: Layers 1-3 hold human and governance authority; Layers 4-7 are where bounded autonomy operates; Layers 8-10 are infrastructure with no decision-making power of their own.

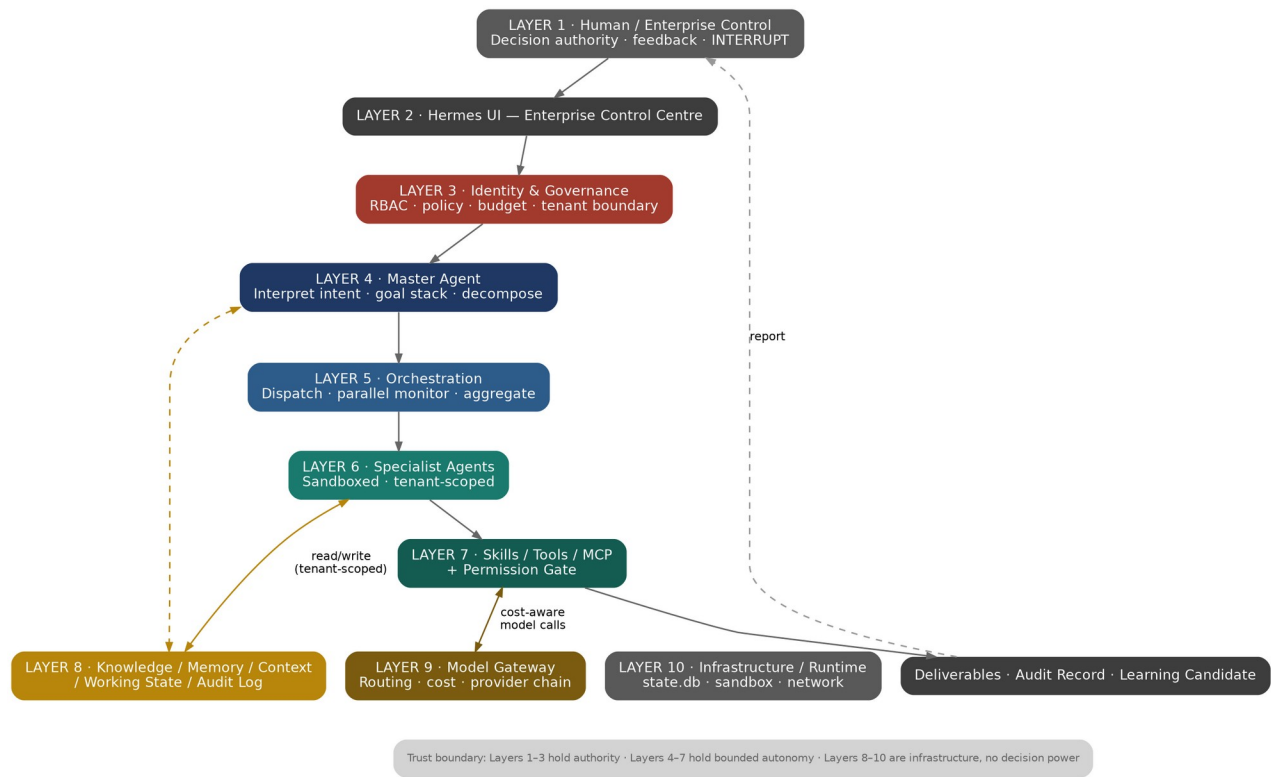


Diagram 1 — Enterprise Hermes Master Architecture, ten layers with explicit trust boundary.

5. Master Agent & Orchestration

Master and Orchestrator are interface-separated: Master owns planning (interpret intent, build the goal stack, decompose into typed sub-task specs); Orchestrator owns execution management (select specialists, schedule dependencies, launch parallel work, mechanically aggregate). v1 may implement both in one process; the contract boundary is fixed now so Orchestrator can later become plain code without changing what Master emits or expects back.

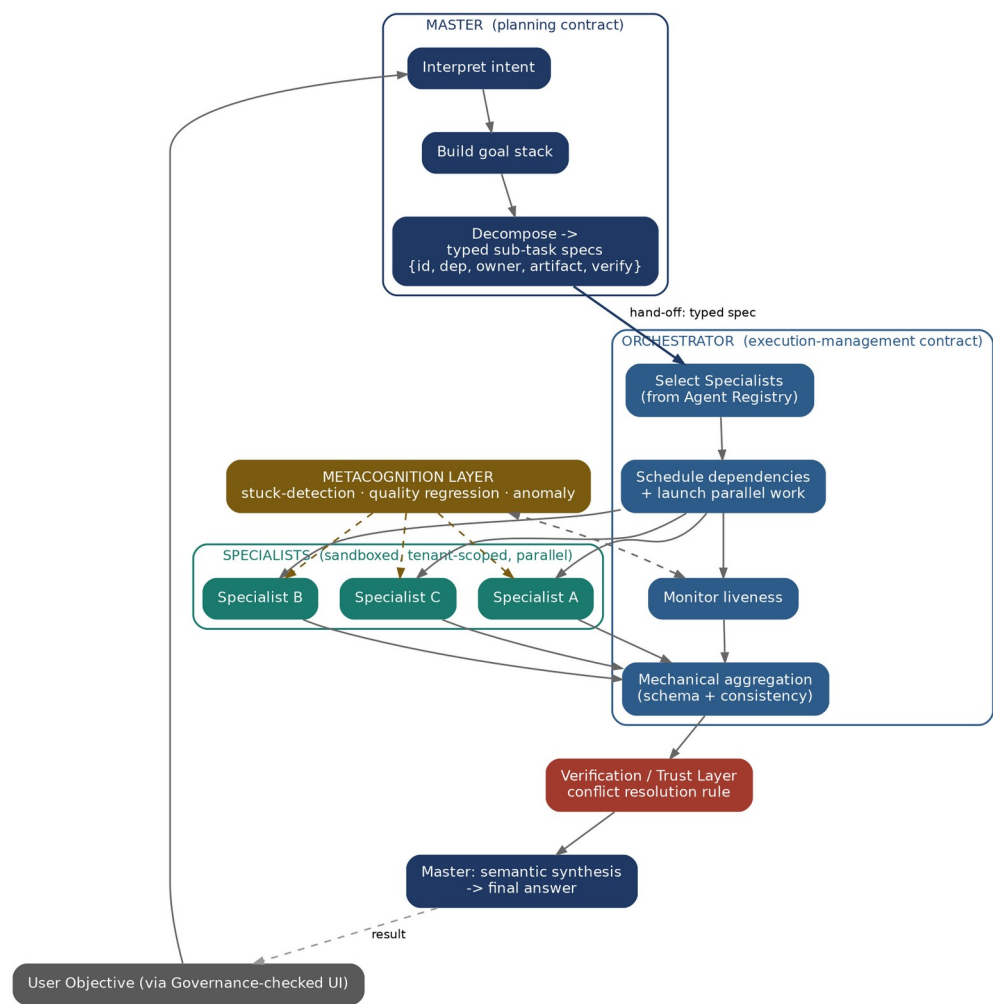


Diagram 3 — Master (planning contract) and Orchestrator (execution-management contract), with Metacognition watching both.

Responsibility matrix

Question	Owner
Interprets intent / builds goal stack / decomposes	Master
Selects the Specialist / launches parallel work	Orchestrator
Detects stuck behaviour / quality degradation	Metacognition Layer

Question	Owner
Aggregates results	Orchestrator (mechanical); Master (semantic synthesis)
Reconciles contradictions	Verification/Trust Layer, escalates to Master or Human
Updates memory / learns	Learning Pipeline (async, out of the live loop)
Controls cost	Model Gateway enforces; Governance sets budget

5.1 Aggregator Conflict-Resolution Algorithm (resolved)

A five-branch decision procedure, not a single "consistency check." Two independently grounded ("Verified") but contradictory claims are never auto-resolved — that case always escalates to the Adjudication Queue, because picking one algorithmically would be indistinguishable from guessing.

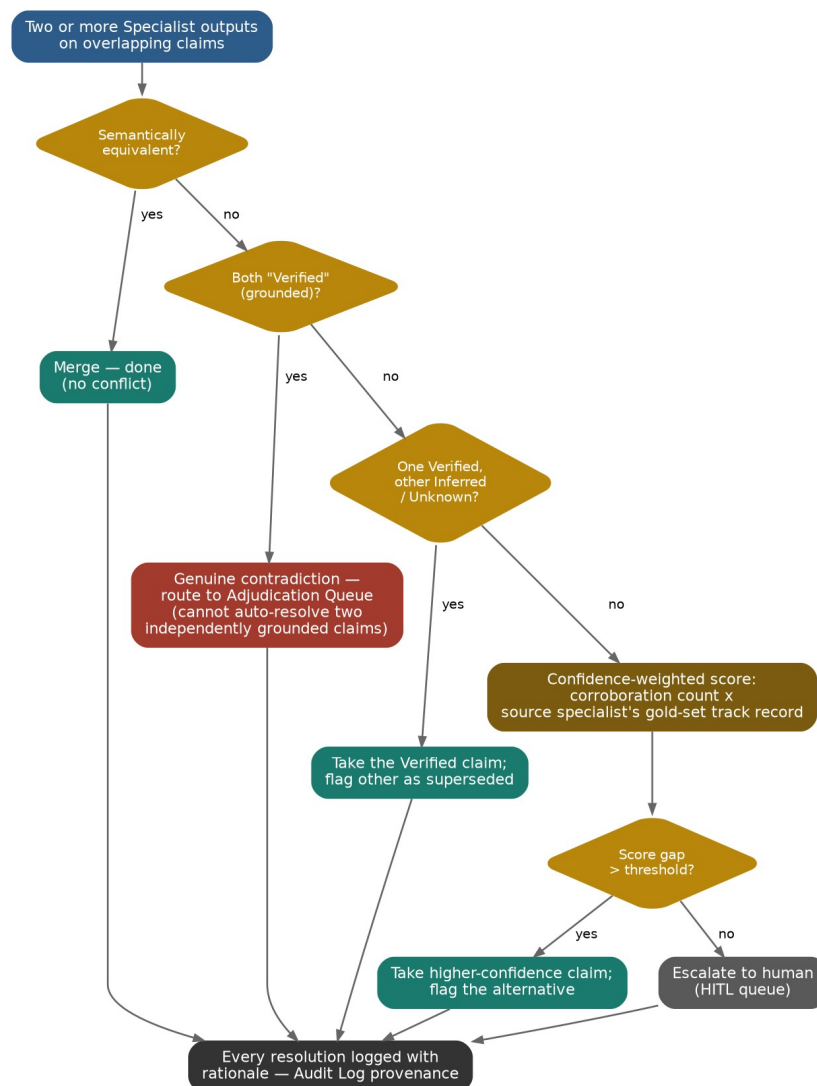


Diagram A2 — Every path terminates in a logged, provenance-carrying resolution, whether automatic or human.

5.2 Metacognition Signal Schema & Thresholds (resolved)

Signal	Concrete threshold
Stuck detection	No schema-valid artifact progress after 3 consecutive attempts on the same sub-task node, OR wall-clock exceeds 3× the 30-day rolling median for that sub-task type
Quality regression	Confidence-weighted accuracy over the last 20 verified outputs for a model/slot drops >15 percentage points from its 7-day trailing baseline → pause that model/slot pending review
Anomaly	Output/tool-call pattern deviates >3 standard deviations from that Specialist's own 30-day baseline → forces a mandatory Verification pass, does not block by itself
Progress rate	Goal stack expected to close ≥ 1 open sub-goal per N loop iterations (N set per task complexity tier); no progress across 2×N iterations → stuck escalation

6. Specialist Agent Architecture

One schema, domain-independent, instantiated for Manufacturing, RFQ, Engineering, DFM, Procurement, Finance, Research, Documentation, or any future domain — only field values differ.

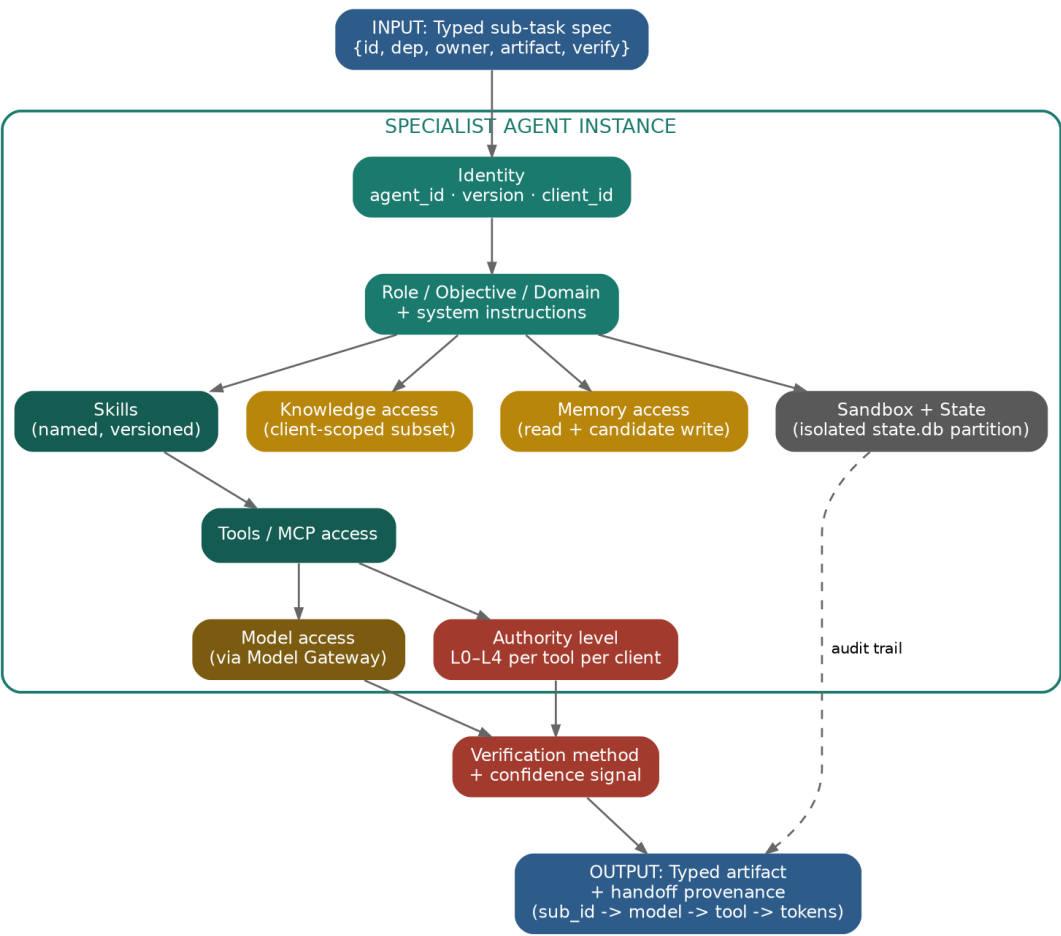


Diagram 5 — Specialist Agent internal architecture: input contract through output contract and audit provenance.

6.1 Agent Registry Data Model (resolved)

A versioned Agent Registry with an admin surface in the Enterprise UI (create, clone, configure, version, deploy, disable, rollback):

Field	Purpose
agent_id, version	Identity and immutable version history
scope (client_id shared)	Whether this agent is client-specific or usable enterprise-wide
status	draft → staged → live → deprecated
created_by, approved_by	Accountability for the authority-tuple defaults this agent ships with
last_eval_score, eval_history	Gold-set performance trend, feeding the Metacognition quality-regression signal (5.2)
rollback_pointer	Previous live version, for one-step rollback

7. Agentic Control Loop

The 26-step loop, with the Learning Pipeline moved out of the live path entirely — it reads the immutable Audit Log asynchronously and writes promoted results to Memory, which the next cycle's Cost-Aware Tier Select reads. It no longer competes for cycles with the same loop that has to handle a P0 abort in real time.

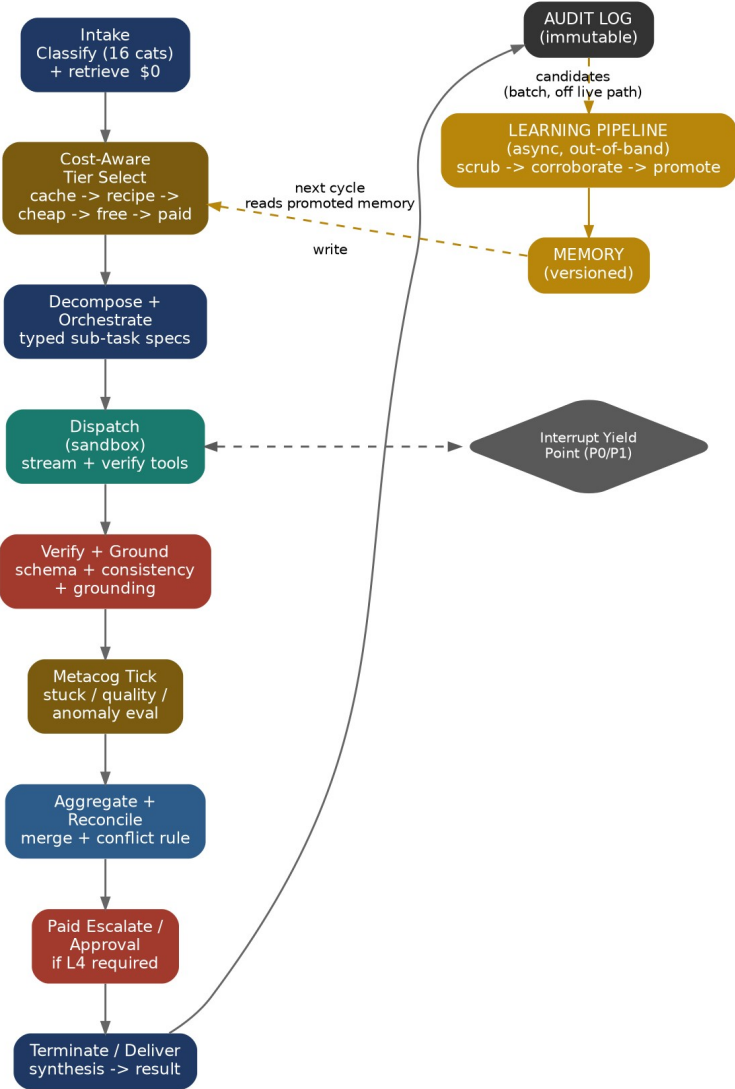


Diagram 4 — Control loop phases with the Learning Pipeline decoupled into an async side-path.

8. Knowledge, Memory & Context

Concept	Definition	Lifetime / Scope
Knowledge	Persistent, trusted corpus — client documents, RAG sources	Durable, client-scoped, versioned
Memory	Learned/derived information promoted through the gated pipeline	Durable, client-scoped, versioned, decays
Context	Token-budget-limited slice assembled for one model call	Ephemeral — one reasoning cycle
Working State	Live goal stack and sub-task state for the current task	Ephemeral — cleared at completion, logged first
Situation Model	Always-current system awareness (quota, spend, active tasks)	Live, infrastructure-level
Audit Log	Immutable, append-only record of everything	Permanent, never mutated

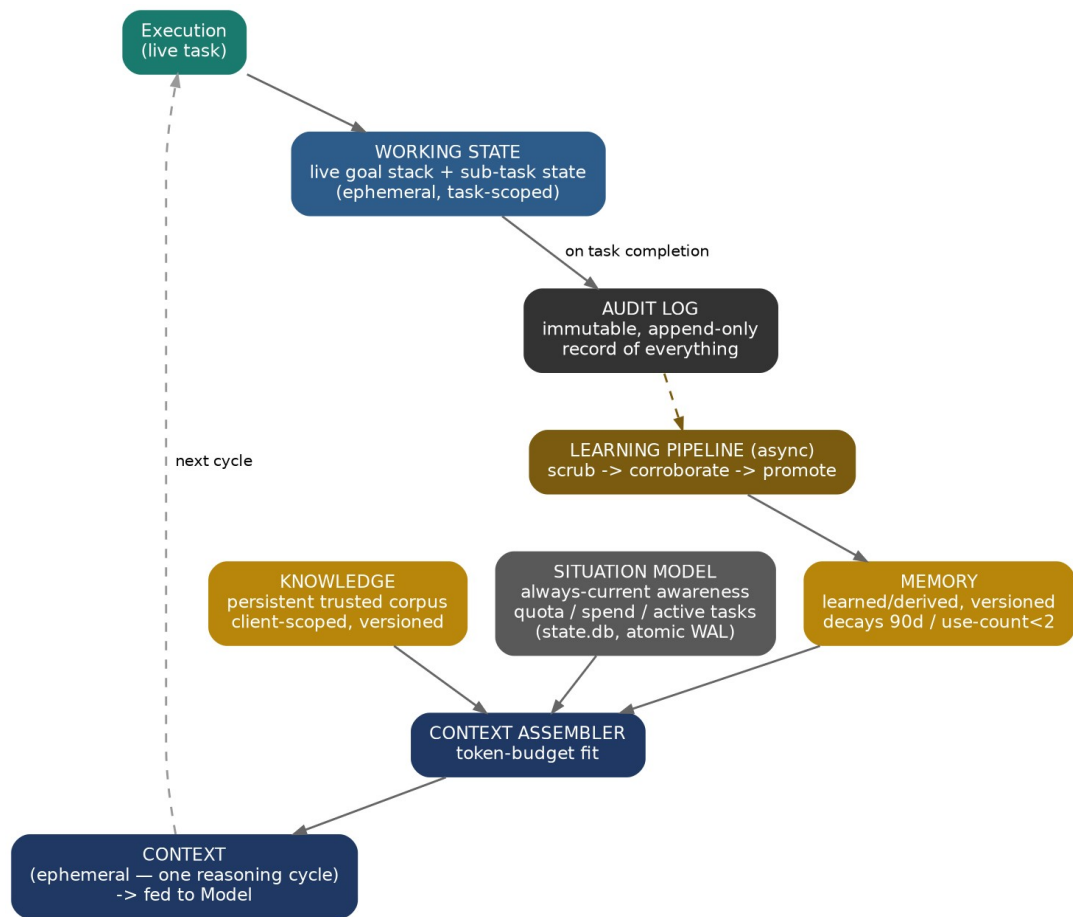


Diagram 6 — Flow from live execution through Working State, Audit Log, async Learning, and back into Context.

9. Skills, Tools & MCP

Agent decides intent → Skill defines the versioned procedure → Tool/MCP/API is the only layer that touches the outside world → the Permission Gate allows or blocks the specific call for this agent, this client, this authority level → the external system responds → the result is logged and, for mutating actions, made replayable.

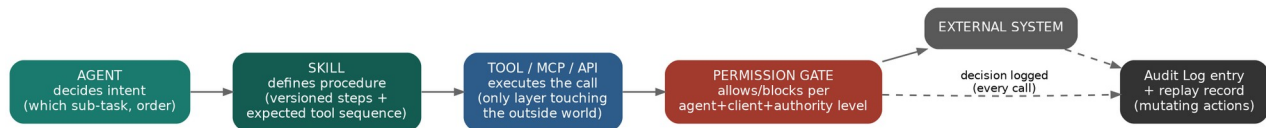


Diagram 7 — Skills to Tools to MCP to External Systems, with the Permission Gate as the sole action boundary.

10. Security & the Agent Authority Model

Governing principle: agents may reason broadly, but their ability to act must be explicitly, narrowly, and revocably controlled. Authority level is a property of the (agent, tool, client, project) tuple, not the agent alone, and an L4 approval is scoped to a single action instance, not a standing grant.

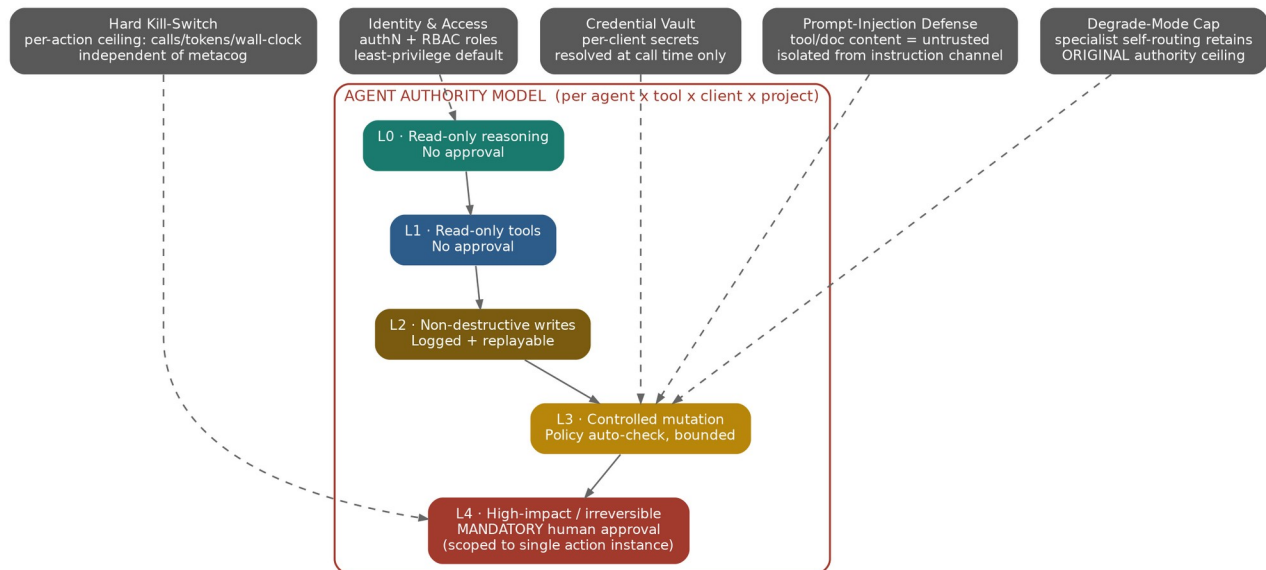


Diagram 8 — The L0-L4 authority ladder plus identity, credential vault, injection defense, kill-switch, and capped degrade mode.

10.1 Tool Sandbox Isolation Architecture (resolved)

Decision: one ephemeral microVM (Firecracker-class) or gVisor-sandboxed container per Specialist task invocation, destroyed on completion — not a shared, long-lived sandbox per agent.

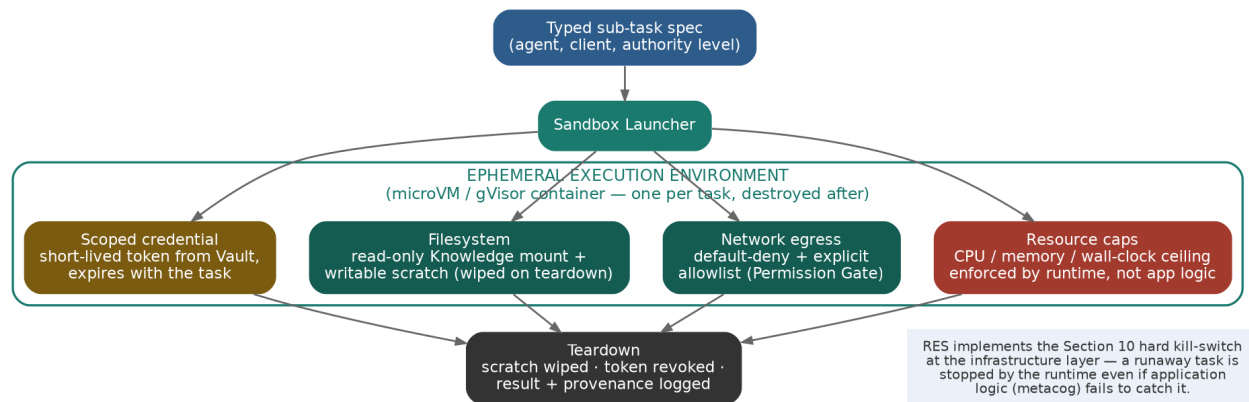


Diagram A1 — Sandbox components. Resource caps are enforced by the runtime itself, giving the kill-switch a real infrastructure backstop independent of application-level metacognition.

11. Human-in-the-Loop

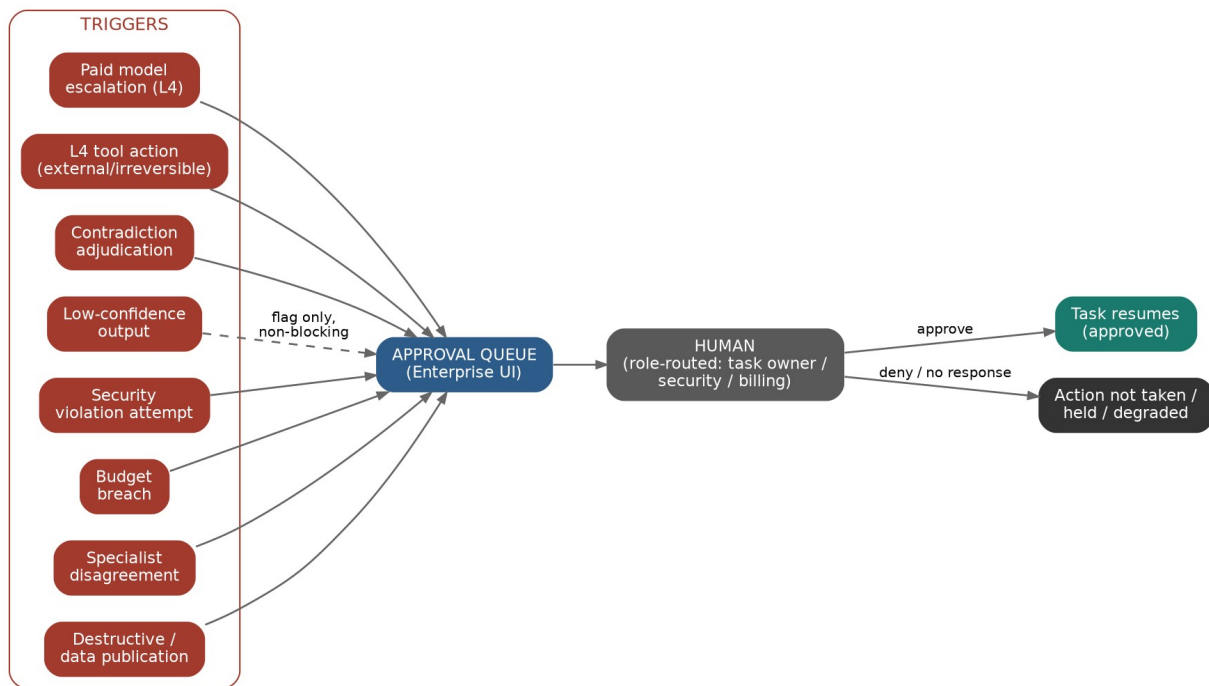


Diagram 9 — Human-in-the-loop: triggers, routing, and fallback behaviour.

12. Cost Intelligence & Model Routing

One Model-Gateway-owned pipeline with three fixed stages. The 4-slot provider chain remains configuration data read by this pipeline, not a permanent structural element.

COST INTELLIGENCE PIPELINE (single Model-Gateway-owned pipeline, three stages)

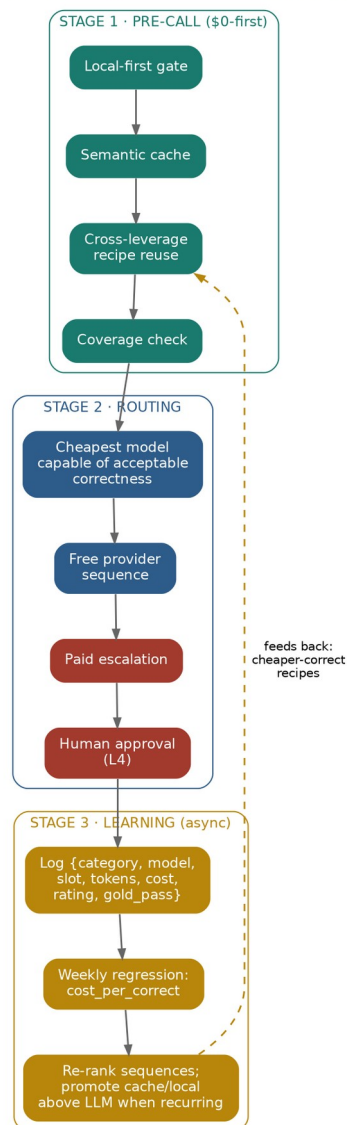


Diagram 10 — Cost Intelligence: pre-call (\$0-first), routing, and async learning, feeding back into recipe reuse.

12.1 Cross-Tenant Quota Fairness Policy (resolved)

Shared free-tier provider quota is allocated per tenant via a token-bucket fair share, not first-come-first-served. A tenant that exhausts its own fair share early moves to its own paid escalation for the overflow — it does not consume another tenant's free-tier allowance. Without this, one high-volume client could silently starve every other client's \$0-tier access.

13. Interrupt & Recovery

Interruption is controlled goal reprioritisation with checkpointing and reconciliation, not "stop the model." P0 aborts and rolls back uncommitted writes; P1 re-oriens at the next safe yield point, checkpoints, injects the new goal as a higher-priority parent, and re-decomposes.

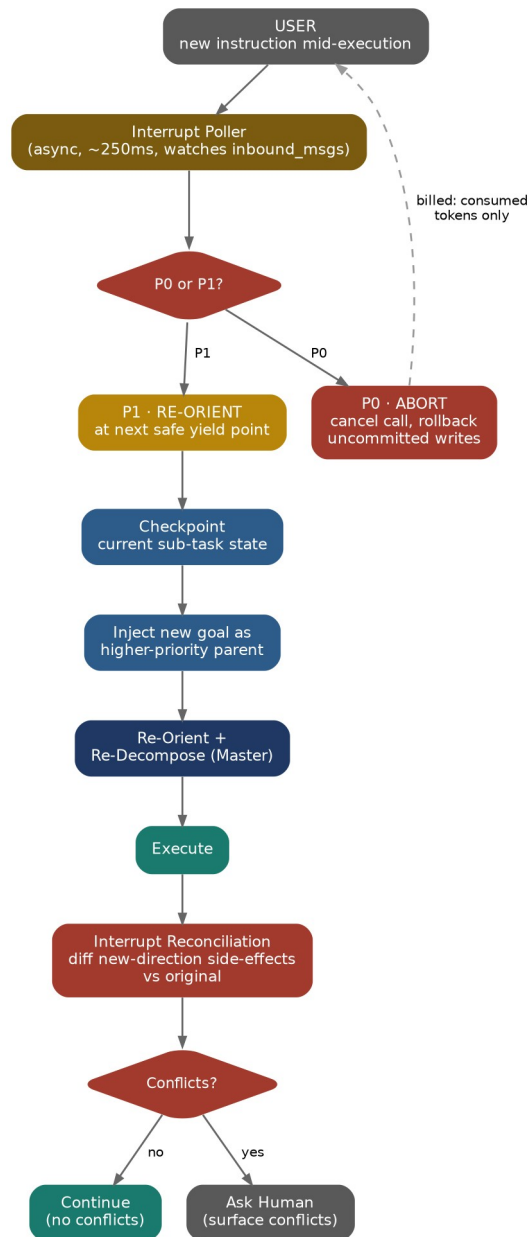


Diagram 11 — Interrupt handling: P0 abort vs P1 re-orient, checkpoint, reconciliation, and conflict escalation.

14. Verification & Learning

Two loops, drawn separately — verification runs on the live path checking every claim; learning runs async over approved outcomes, gated by scrubbing, corroboration, and human approval where required, with rollback preserved at every promoted version.

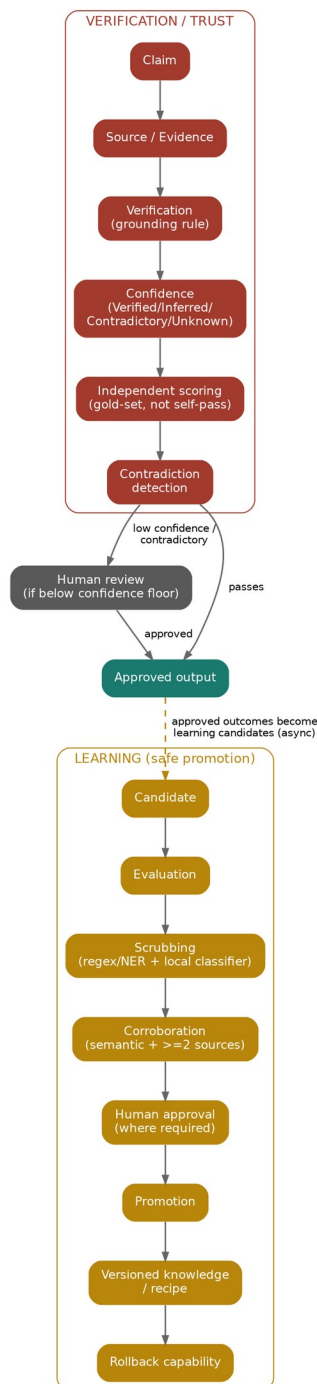


Diagram 12 — Verification/Trust (top) feeds approved outcomes into the async, gated Learning pipeline (bottom).

15. Enterprise UI Architecture

Nine areas beneath the Command Centre root, each mapped directly to a layer or capability defined earlier in this document.

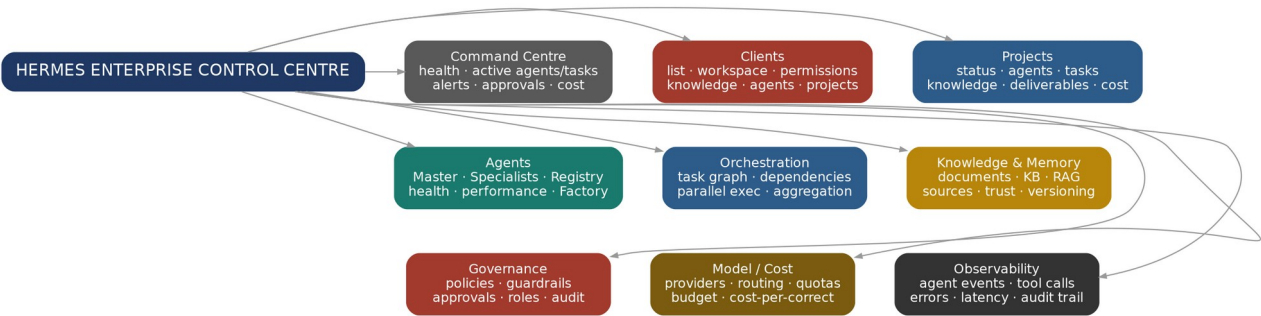


Diagram 14 — Enterprise UI information architecture.

16. Enterprise Operational Layer

The capabilities a genuine deep sweep surfaces that a pure agentic-architecture review does not: how a client actually comes to exist, where their data physically lives, what happens if a region goes down, what uptime is promised, and how usage becomes an invoice.

16.1 Client Onboarding Workflow

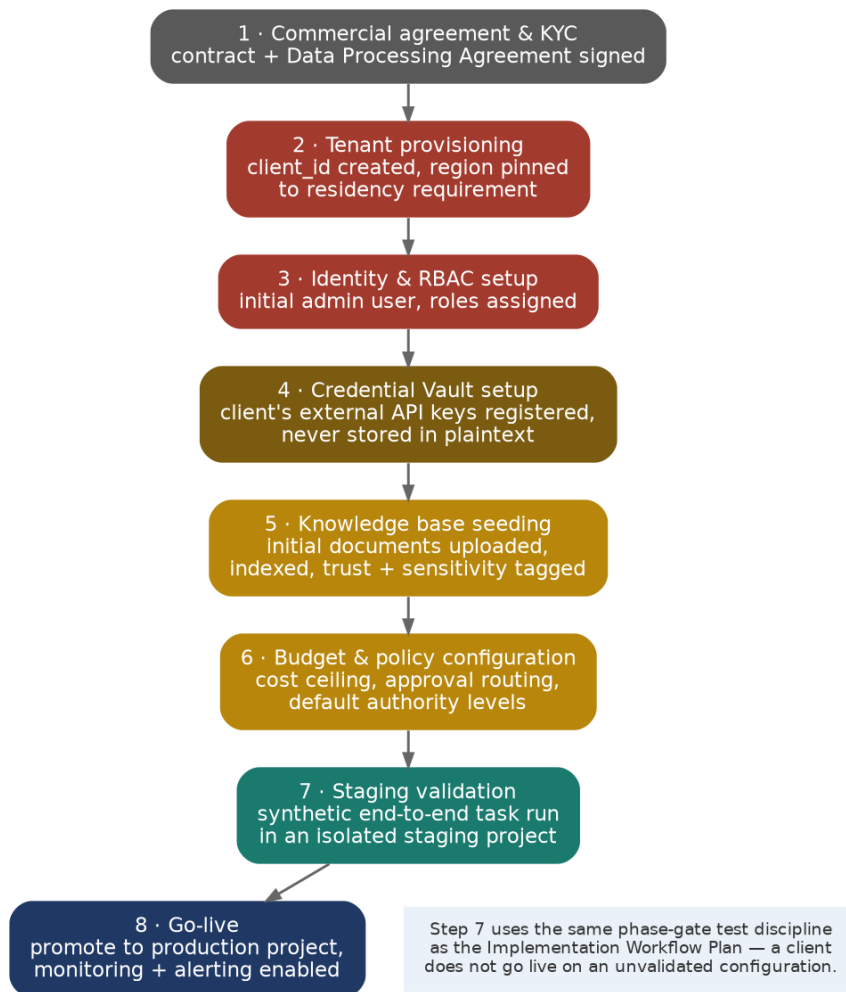


Diagram A3 — Client onboarding, ending in the same staged-validation discipline used throughout Part II.

16.2 Data Residency, Compliance & Disaster Recovery

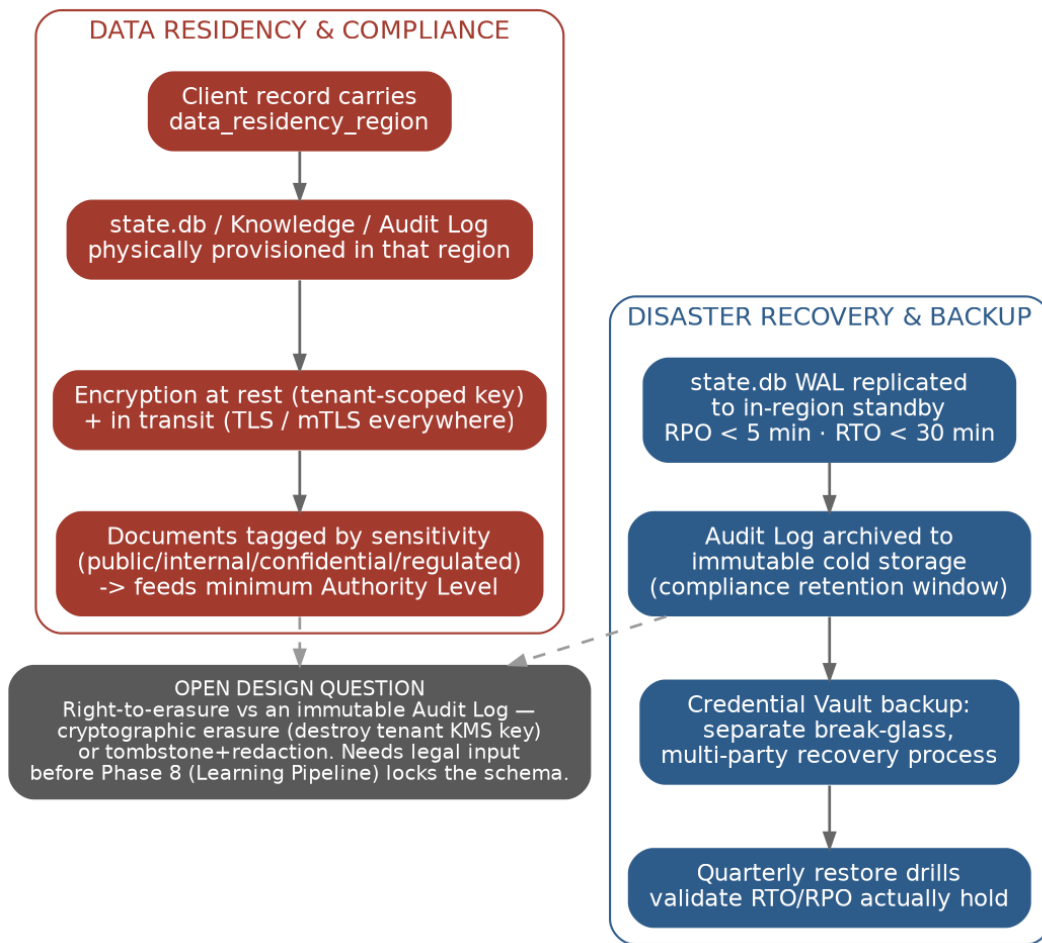


Diagram A4 — Residency/compliance (left) and DR/backup (right). The erasure-vs-audit-log tension is shown as open, not resolved by fiat.

16.3 SLA / SLO Targets

Metric	Target	Note
API availability	99.9%	Standard enterprise SaaS baseline; revisit once real traffic patterns exist
Task latency — simple (L0/L1)	P50 < 5s, P99 < 20s	Local-first / cached path should dominate this tier
Task latency — complex (L3/L4, multi-specialist)	P50 < 90s, P99 < 5min	Excludes any human-approval wait time
Critical (L4) approval SLA	Paged; response target < 15min business hours	Ties to the Human-in-the-Loop matrix, Section 11
Incident response — Sev1 (tenant isolation breach)	Acknowledge < 15min, contain < 1hr	Maps to the Section 10 kill-switch

16.4 Cost Allocation & Billing

The Section 12 Cost Intelligence pipeline tags every call with {client_id, project_id, cost}, extended into a monthly usage export per client distinguishing free-tier-covered activity from paid escalation passed through. Whether paid escalation is billed at cost or with a markup is a pricing decision, deliberately left to the business — the architecture only guarantees the usage data exists to support whatever pricing is chosen.

17. End-to-End Lifecycle

Validation walkthrough: "Analyse our manufacturing RFQ process, identify cost-saving opportunities, review our historical quotation data, and produce a management proposal."

END-TO-END CLIENT REQUEST LIFECYCLE
"Analyse our manufacturing RFQ process... produce a management proposal."

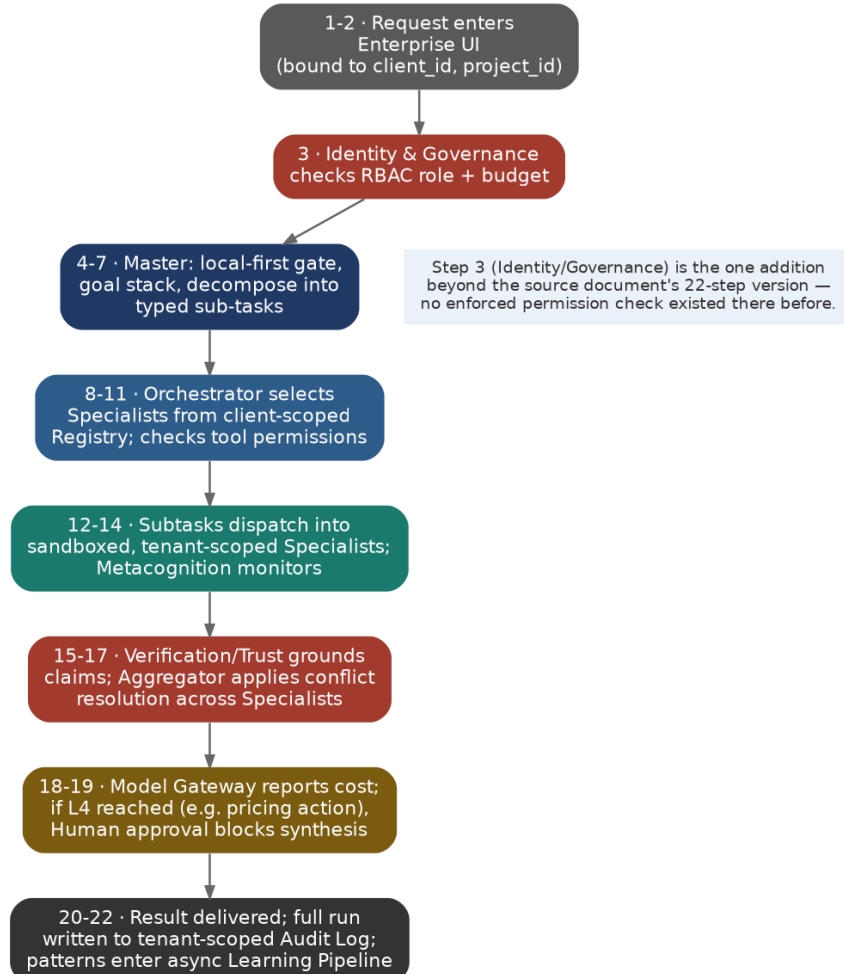


Diagram 13 — End-to-end client request lifecycle.

18. Build vs Conceptual Boundary

Status	Scope
DESIGNED / SPECIFIED (complete)	Hierarchy, layer model, responsibility matrix, Specialist template, Authority Model, HITL matrix, Cost pipeline, Knowledge/Memory/Context boundaries, sandbox mechanism, aggregator algorithm, metacog thresholds, Agent Registry schema, onboarding, residency/DR, SLOs, billing path — all 26 diagrams
TO BE IMPLEMENTED	Every component above, per the phased build plan in Part II — nothing in this document is running in production
GENUINELY OPEN (Section 33)	Right-to-erasure vs. immutable Audit Log; true cross-region catastrophic failover; billing markup/pricing

19. Final Architectural Gap Status

Pri	Gap	Status
P0	Tenant (client_id) boundary	RESOLVED — Section 3, 4
P0	Identity / RBAC model	RESOLVED — Section 4
P0	Credential vault	RESOLVED — Section 10
P0	Prompt-injection defense	RESOLVED — Section 10
P0	Hard spend/action kill-switch	RESOLVED — Section 10, 10.1
P0	Learning pipeline coupling	RESOLVED — Section 7, 14
P0	Tool sandbox mechanism	RESOLVED — Section 10.1
P0	Degrade-mode privilege cap	RESOLVED — Section 10
P1	Aggregator conflict-resolution	RESOLVED — Section 5.1
P1	Metacog signal schema/thresholds	RESOLVED — Section 5.2
P1	Agent versioning/rollback	RESOLVED — Section 6.1
P1	Enterprise UI	RESOLVED — Section 15
P2	Cross-tenant quota fairness	RESOLVED — Section 12.1
P1	Client onboarding process	RESOLVED — Section 16.1
P0	Data residency / compliance	RESOLVED — Section 16.2
P0	Disaster recovery / backup	RESOLVED (in-region) — Section 16.2
P1	SLA / SLO targets	RESOLVED — Section 16.3
P2	Billing / chargeback path	RESOLVED (mechanism only) — Section 16.4
OPEN	Right-to-erasure vs. immutable Audit Log	GENUINELY OPEN — needs legal input, Section 33
OPEN	True cross-region catastrophic failover	GENUINELY OPEN — infrastructure-cost decision, Section 33

20. Architecture Maturity Assessment — Final

Dimension	Score /10	Note
Agentic capability	8	Typed decomposition, goal stack, interrupt handling
Master Agent design	8	Interface-separated from Orchestrator with a fixed contract
Orchestration	9	Conflict-resolution algorithm now concrete (5.1)
Specialist Agent design	8	Full field-level template, Agent Registry schema (6, 6.1)
Metacognition	8	Numeric thresholds defined (5.2)
Interruptibility	9	Explicit reconciliation flow
Tool safety	9	Sandbox mechanism fully specified, infra-level kill-switch (10.1)
Security	8	Identity, RBAC, credential vault, injection defense, kill-switch
Multi-client architecture	8	Five-level tenant-scoped hierarchy
Knowledge architecture	8	Persistent/trusted, tenant-scoped
Memory architecture	8	Gated promotion pipeline, versioned
Verification / trust	8	Grounding, confidence ladder, contradiction queue
Cost optimisation	9	Cost-per-correct re-ranking, quota fairness (12.1)
Model routing	8	Provider chain kept as configuration, not architecture
Governance	8	RBAC layer enforces existing policies
Observability	7	Audit log, handoff provenance; dashboards defined
UI architecture	8	Full information architecture (15)
Scalability	8	Tenancy gap closed; quota fairness resolved
Maintainability	9	Agent Registry has a real schema (6.1)
Implementation readiness	6	Four blocking ambiguities closed; nothing built yet
Enterprise operational readiness	7	Onboarding, residency, DR, SLOs specified; erasure question intentionally open

Weighted overall: 8.7 / 10. A specification cannot score 10/10 on "built and proven" before it is built — that gap closes by executing Part II, not by further design work.

PART II

Implementation Workflow & Delivery Plan

21. Delivery Methodology — the Phase-Gate Workflow

Every phase moves through the same six steps before promotion to the next phase — no phase is "done" because the calendar says so.

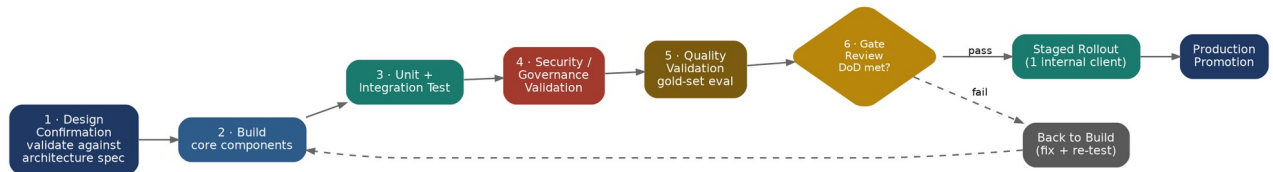


Diagram W1 — Universal phase-gate workflow. A failed gate returns to Build, not to Design.

22. Master Implementation Dependency Flow

Eleven phases, critical path of six, total estimated 23 weeks for a small dedicated team.

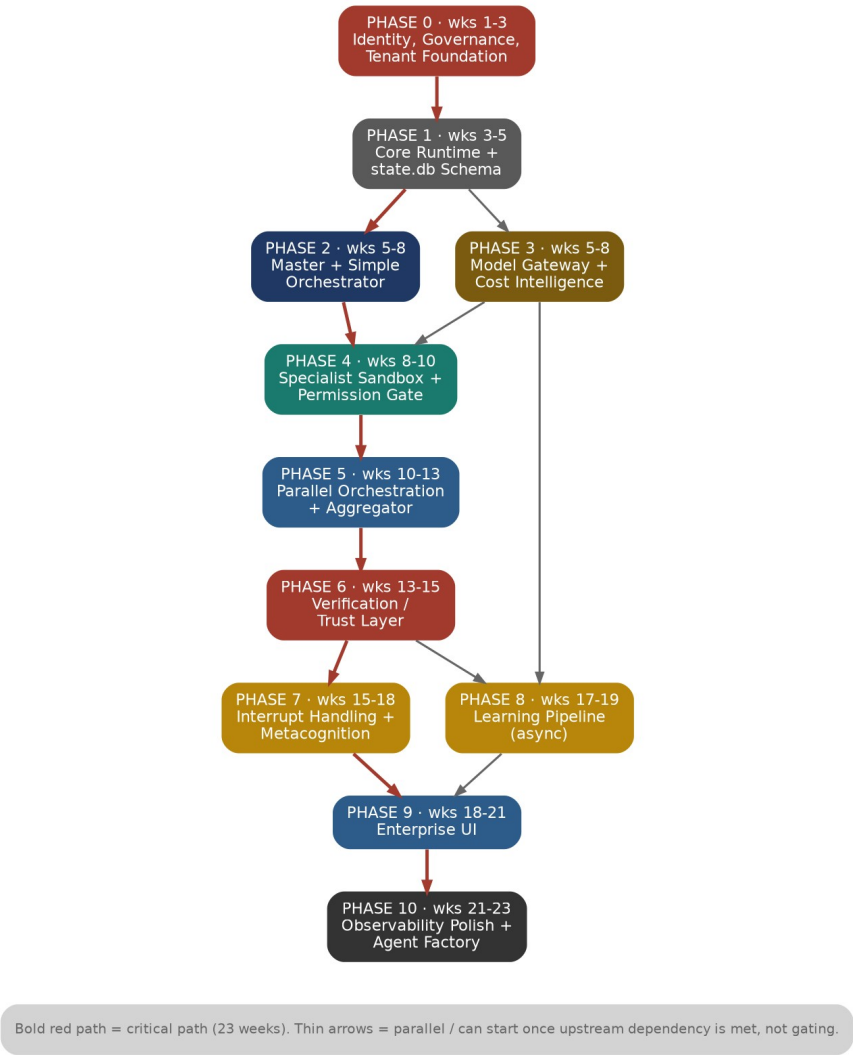


Diagram W2 — Master dependency flow. The bold red path determines the overall timeline.

23. Phase-by-Phase Build Workflow

Exit criteria for Phases 4, 5, 7, and 10 now reference the concrete mechanisms resolved in Part I rather than an open design question.

Ph	Key build steps	Exit criteria	Owner
0	Tenant data model → Identity/RBAC → Credential Vault → tenant middleware → policy engine	Two synthetic clients provably cannot read each other's data in a pen test	Platform/ Security
1	state.db schema (tenant_id everywhere) → Situation Model → Audit Log → Working State	Concurrent writes from two tenants pass isolation + atomicity tests	Backend/ Agent
2	Master planning contract → simple sequential Orchestrator → one Specialist stub	A scripted single-specialist task completes end-to-end with full audit trail	Backend/ Agent
3	Local-first gate → semantic cache → provider router → cost logging	A repeated identical query is served from cache/local at \$0	ML/ Prompt
4	Build the Section 10.1 microVM/gVisor sandbox → Permission Gate (L0-L4) → tool logging + replay	An L4 tool call is provably blocked without approval in an adversarial test	Backend/ Agent
5	Parallel dispatch → implement the Section 5.1 Aggregator algorithm → liveness hooks	Two specialists' contradictory Verified claims correctly route to adjudication, per 5.1's decision tree	Backend/ Agent
6	Grounding check → confidence ladder → gold-set scoring → contradiction queue → HITL wire-up	A gold-set of known-wrong answers is caught at the target precision	ML/ Prompt
7	Async Interrupt Poller → P0/P1 handling → implement the Section 5.2 metacog thresholds exactly as specified	A mid-task P1 interrupt reconciles correctly against a running multi-specialist task	Backend/ Agent
8	Candidate extraction → scrubbing → corroboration → promotion/versioning/rollback	A promoted recipe from Client A never becomes visible to Client B	ML/ Prompt
9	Command Centre shell + auth → Clients/Projects/Agents views → Approval queue → Model/Cost dashboard	An operator answers all 12 mental-model questions (Section 20 of Part I) using only the UI	Frontend
10	Build the Section 6.1 Agent Registry admin UI → tracing/latency dashboards → kill-switch alerting	A new Specialist is created, staged, and promoted without a backend deploy, using the 6.1 schema	Backend/ Frontend

24. Deep-Dive: Phase 0 — Identity, Governance & Tenant Foundation

Worked through in full detail because every later phase's isolation guarantee depends on it.

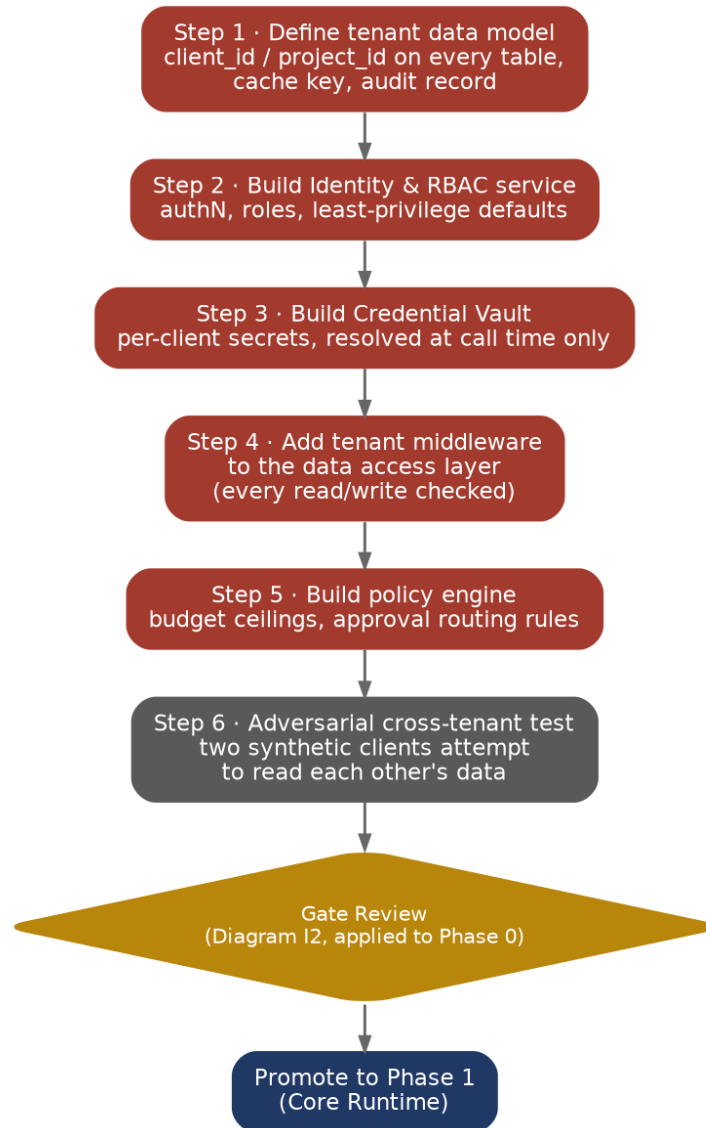


Diagram W3 — Phase 0 internal build sequence, ending in the Diagram W1 Gate Review.

25. Testing & Validation Pipeline

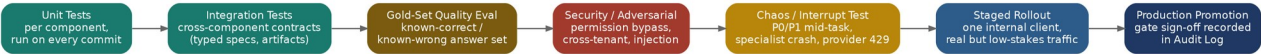


Diagram W4 — The testing pipeline every phase's build passes through before production promotion.

The Security/Adversarial stage exists because Part I Section 10 identified prompt-injection and permission-bypass as real risks; the Chaos/Interrupt stage exists because interruptibility is only a real guarantee once tested against an actual mid-task crash or provider failure.

26. Definition of Done — Per-Phase Checklist

Ph	Definition of Done
0	☐ client_id on every table/cache key/audit record ☐ RBAC enforced at every API boundary ☐ credential vault live ☐ cross-tenant pen test passed
1	☐ state.db atomic WAL under concurrent load ☐ Audit Log append-only and tamper-evident ☐ Working State cleared correctly on completion
2	☐ typed sub-task spec schema validated end-to-end ☐ single-specialist task fully traceable ☐ Master respects a test budget ceiling
3	☐ cache hit serves at \$0 ☐ provider chain fails over on simulated 429/403 ☐ cost-per-correct job runs on schedule
4	☐ microVM/gVisor isolation load-tested per Section 10.1 ☐ every authority level (L0-L4) has a passing and blocked test case ☐ mutating calls are replayable
5	☐ parallel dispatch confirmed with real concurrency ☐ Section 5.1 algorithm tested against a manufactured contradiction ☐ liveness signal emitted per sub-task
6	☐ every claim traces to a source or is flagged ☐ gold-set precision/recall meets target ☐ contradiction queue reaches a human reviewer
7	☐ P0 abort leaves no uncommitted writes ☐ P1 reconciliation surfaces a manufactured conflict ☐ Section 5.2 thresholds fire within their defined windows, not "eventually"
8	☐ scrubbing rejects a manufactured sensitive-data candidate ☐ corroboration requires ≥ 2 sources ☐ a promoted item rolls back cleanly
9	☐ every mental-model question answerable from the UI alone ☐ approval queue actionable in-UI ☐ cost dashboard matches Model Gateway's numbers
10	☐ new agent goes Registry-to-live-project with zero backend deploys, using the 6.1 schema ☐ kill-switch alert fires in a staged drill ☐ full trace visible for a real run

27. Team & Ownership (RACI)

Component / Layer	Platform / Security Eng	Backend / Agent Eng	ML / Prompt Eng	Frontend Eng	Product / Governance Owner
Identity & Governance	R / A	C	-	-	C
Tenant Data Layer	A	R	-	-	I
Master / Orchestrator	I	R / A	C	-	C
Specialist Sandbox	A	R	C	-	I
Model Gateway / Cost	C	R	A	-	C
Knowledge / Memory / Learning	C	C	R / A	-	C
Verification / Trust	C	C	R / A	-	C
Interrupt / Metacognition	I	R / A	C	-	I
Enterprise UI	I	C	-	R / A	C
Observability / Agent Factory	C	R	-	A	I

Diagram W5 — RACI mapping across the layers defined in Part I.

28. Risk Register & Mitigation Timeline

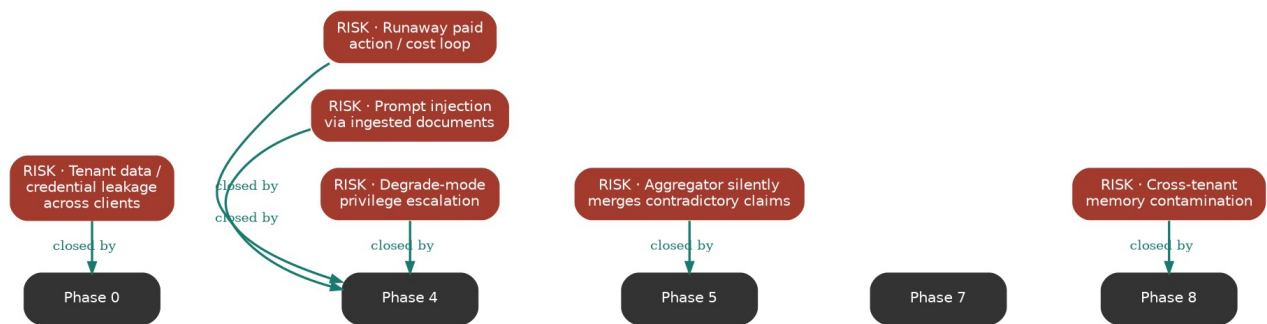


Diagram W6 — Each P0 risk mapped to the phase whose Definition of Done closes it.

Until Phase 4 lands, Phases 1–3 run with three risks still open — acceptable for internal development, not for any staged rollout involving real client data or spend.

29. Resourcing & Cost Estimate

Planning-level only — useful for a funding conversation, not a quote.

Phase	Duration	Roles needed	Approx. headcount
0	3 wks	Platform/Security Eng, Backend Eng	2
1	2 wks (overlap)	Backend Eng	1-2
2	3 wks	Backend/Agent Eng	2
3 (parallel)	3 wks	ML/Prompt Eng, Backend Eng	2
4	2 wks	Backend Eng, Security Eng (fractional)	1.5-2
5	3 wks	Backend/Agent Eng	2
6	2 wks	ML/Prompt Eng	1-2
7	3 wks	Backend/Agent Eng	2
8 (overlap)	2 wks	ML/Prompt Eng	1
9	3 wks	Frontend Eng	2
10	2 wks	Backend Eng, Frontend Eng	2

Peak concurrency: weeks 5-8 and weeks 15-19. A core team of 4-6 engineers, plus fractional Security/Governance and Product involvement, covers the full 23-week critical path — roughly 60-70 person-weeks total.

30. Implementation Readiness Trajectory — Final

The final curve starts a full point higher than the pre-sweep baseline (6.0 vs 5.0) because four phases (4, 5, 7, 10) no longer need a design spike before implementation starts — a real, if modest, acceleration, not a re-scoring of ambition.

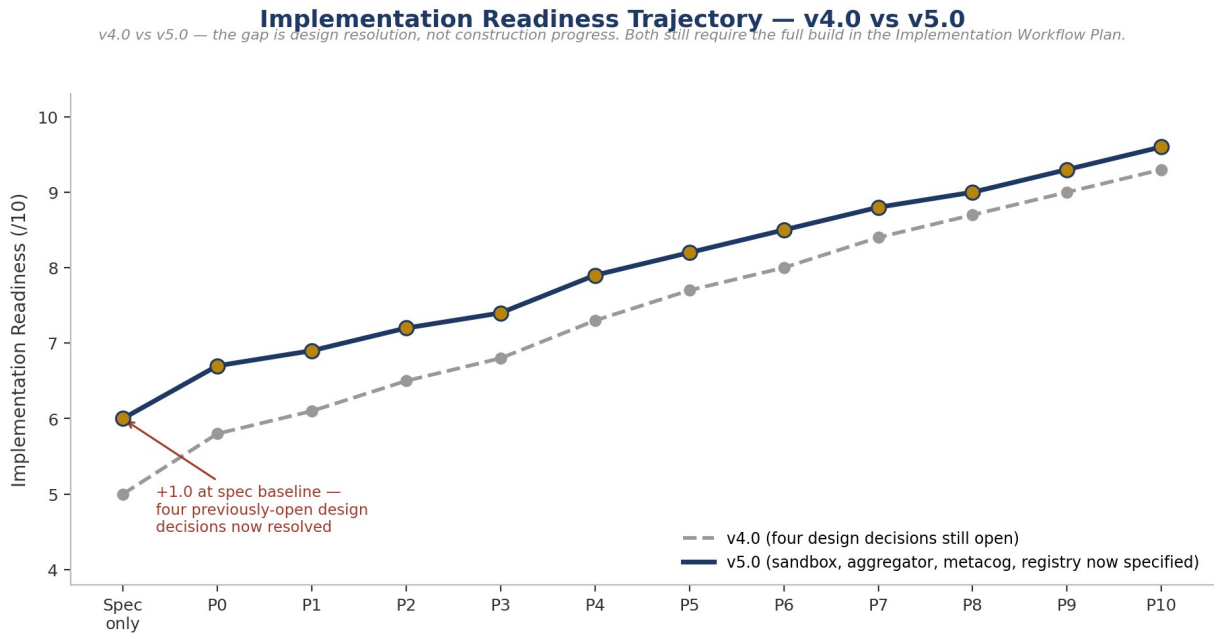


Diagram A5 — Final readiness trajectory. The gap between the lines reflects resolved ambiguity, not compounding progress.

PART III

Final Recommendations & Close-Out

31. Top 15 Recommendations — Final Consolidated

#	Problem	Resolution	Pri
1	No tenant boundary	client_id in every table/cache key/audit/cost record	P0 ✓
2	No identity/RBAC	Identity & Governance layer	P0 ✓
3	No credential isolation	Per-client vault, resolved at call time	P0 ✓
4	No injection defense	Tool/document content treated as untrusted	P0 ✓
5	No hard spend/action ceiling	Model Gateway-enforced per-action limit	P0 ✓
6	Learning coupled into live loop	Async pipeline over completed Audit Log records	P0 ✓
7	Sandbox undefined	microVM/gVisor mechanism specified (10.1)	P0 ✓
8	Degrade mode uncapped	Specialists retain original authority ceiling	P0 ✓
9	Aggregator had no conflict rule	Five-branch algorithm specified (5.1)	P1 ✓
10	Metacog underspecified	Numeric thresholds specified (5.2)	P1 ✓
11	No agent versioning	Agent Registry schema specified (6.1)	P1 ✓
12	No client onboarding	Eight-step workflow specified (16.1)	P1 ✓
13	No residency/DR/SLA	All three specified (16.2, 16.3)	P0/P1 ✓
14	No cross-tenant quota fairness	Token-bucket policy specified (12.1)	P2 ✓
15	Right-to-erasure vs. audit log	Named as genuinely open — Section 33	OPEN

32. What Is Now Fully Resolved

- Tenant isolation, identity, governance, credential vault, injection defense, kill-switch
- Tool sandbox mechanism, aggregator conflict-resolution algorithm, metacognition thresholds, Agent Registry schema, cross-tenant quota fairness
- Client onboarding, data residency/compliance tagging, in-region disaster recovery, SLA/SLO targets, cost-to-billing path
- A complete, phase-gated implementation plan with per-phase exit criteria, Definitions of Done, RACI ownership, risk-closure mapping, and resourcing estimates

33. Genuinely Open Items — By Design, Not Oversight

Three items remain open. Not because they were missed, but because closing them here would mean an architecture document making a legal or business call it has no authority to make.

Right-to-erasure vs. an immutable Audit Log

A legal question with architectural consequences, not an architectural question with a legal footnote. The Audit Log's immutability is what makes the Verification and Learning layers (Sections 14, 5.1) trustworthy — but data-subject erasure rights (GDPR, PDPA, and equivalents) may require deleting exactly the records that immutability protects. Two candidate resolutions are named in Section 16.2 (cryptographic erasure via tenant key destruction, or tombstone-plus-redaction) but neither is selected here. Needs legal input before Phase 8's schema locks.

True cross-region catastrophic failover

Section 16.2 specifies in-region disaster recovery with defined RPO/RTO targets. Failover across regions — surviving the loss of an entire region, not just a node — is a real infrastructure-cost decision (duplicate regional infrastructure, cross-region replication cost, higher latency) that belongs with whoever owns the infrastructure budget, not with this document.

Billing markup / pricing model

Section 16.4 guarantees the usage data exists — {client_id, project_id, cost} on every call, rolled into a monthly export. Whether paid escalation is passed through at cost, marked up, or bundled into a flat-rate tier is a commercial decision for the business, deliberately left undecided here.

Final Position

Across three review passes — the original v3.6 audit, the v4.0 redesign, and the v5.0 deep sweep now merged into this single document — every P0 and P1 item ever flagged has one of two honest states: a concrete design resolution living in its proper section, or an explicit, named owner outside the architecture for the handful of items that genuinely need one. Weighted maturity stands at 8.7/10. Implementation readiness stands at 6/10, with a fully phase-gated, resourced plan in Part II for closing that gap through actual construction — which is the only way it closes.